PROFELINK
DIVISIÓN DE TRABAJO

Guía paso a paso para cuatro integrantes

Proyecto Final Parte 2 | Evaluaciones en vivo tipo Kahoot

Paradigmas de Programación · EIF400 · II Ciclo 2026

OBJETIVO DEL DOCUMENTO
Que las cuatro personas puedan desarrollar en paralelo, saber exactamente qué hacer primero y cómo demostrar que cada tarea funciona, sin duplicar código ni depender constantemente de otros compañeros.

Decisión de trabajo	Propuesta inicial
Frontend	React + TypeScript
Backend del juego	Una API NestJS modular, REST + Socket.IO
Identidad	Supabase Auth; validación JWT en NestJS
Persistencia	PostgreSQL y almacenamiento persistente para imágenes
Organización	Cuatro responsables con entregables verificables

Documento de planificación; no certifica funcionalidades implementadas. Versión para revisar en reunión inicial del equipo.

0. Guía rápida de lectura

Este material está pensado para enviarlo al grupo. La sección 1 fija las reglas; la 2 establece lo que exige la entrega; la 3 define las decisiones comunes; las secciones 4 a 7 describen el trabajo por persona; las secciones posteriores explican el orden, las pruebas, la integración y el día de la exposición.

CONDICIÓN POR CONFIRMAR CON EL PROFESOR
El profesor autorizó Supabase Auth como tecnología, pero el enunciado también pide un servicio de autenticación implementado como proyecto/API separado y audiencia dirigida a la API del juego. Antes de cerrar el diseño, confirmar expresamente si Supabase Auth externo sustituye esa API propia. Si no la sustituye, se añade una Auth API pequeña como trabajo transversal y se reajusta la carga.

Mapa del trabajo

Persona	Área principal	Producto visible
P1	Datos + backend REST + imágenes	CRUD de cuestionarios seguro y persistente
P2	Salas + juego + WebSockets	Partida simultánea con reglas confiables
P3	Frontend de autor/anfitrión + login	Crear quiz, organizar e iniciar partida
P4	Frontend participante + integración	Unirse, responder, reconectar y ejecutar la demo

Qué no significa esta división

- Nadie queda encargado solamente de informes, presentaciones o infraestructura: los cuatro entregan funcionalidad de la aplicación.
- «Dueño» de un módulo significa que coordina ese código, no que nadie más puede ayudar. Las integraciones son tareas compartidas.
- No se debe programar todo por separado y unirlo al final: se integra un flujo mínimo desde el inicio.
- Los ejemplos y nombres de eventos propuestos son contratos de planificación, no implementaciones ya verificadas.

1. Reglas del equipo: aplican a los cuatro

1. Un responsable claro por funcionalidad. Cada tarea tiene un dueño y un revisor diferente. Se especifica quién implementa, quién consume su contrato y quién valida.

2. Trabajo poco a poco. Un PR debe resolver una funcionalidad pequeña: por ejemplo crear un quiz, no «todo el backend». Las tareas grandes se dividen y se comprueban por separado.

3. Acuerdos antes de programar. Primero se pactan entidades, rutas REST, eventos WebSocket, payloads, estados y respuestas de error. El frontend puede usar mocks si el contrato ya es estable.

4. Nunca romper el trabajo de otra persona. No cambiar rutas, DTO, estructura JSON, nombres de eventos ni migraciones compartidas sin avisar y actualizar las especificaciones.

5. Una tarea no termina cuando compila. Termina cuando pasa pruebas, se demuestra con un caso reproducible, tiene documentación y fue integrada a la rama común.

6. Probar éxitos y fallos. Cada tarea debe cubrir caso normal, dato inválido, falta de permisos y al menos un error de conexión o concurrencia cuando corresponda.

7. Git con ramas y PR. Rama por tarea; mensajes claros; PR pequeño con objetivo, cambios, capturas/requests, tests, riesgos y OpenSpec final archivado cuando aplique.

8. Evitar trabajos aislados. Reunión breve de integración frecuente. Si un contrato bloquea a otra persona, se prioriza resolverlo antes de agregar nuevas funciones.

9. Seguridad primero. Sin secretos en Git, ni credenciales en capturas. NestJS valida tokens y permisos; React nunca es autoridad para puntajes, tiempos o control de salas.

10. Todos conocen el recorrido completo. Cada integrante puede explicar registro, quiz, sala, respuesta, puntaje, persistencia y despliegue, aunque su especialidad sea otra.

Regla de aceptación («definición de terminado»)

1. El contrato acordado está en OpenSpec o el documento de contratos compartido.
2. La implementación funciona con datos reales o con un mock claramente identificado.
3. Hay al menos una prueba automatizada relevante y una prueba manual explicada.
4. Los errores de acceso, validación y conexión se muestran o responden correctamente.
5. El código pasó revisión, compilación y se integró sin romper el flujo principal.
6. Se dejó evidencia en el PR: comando, resultado, captura o video corto, sin datos sensibles.

Analogía
Piensen en una construcción: una persona levanta la estructura, otra instala el sistema eléctrico, otra diseña los espacios y otra prueba las conexiones y acabados. Ninguna termina su parte si la puerta no encaja con el marco que hizo su compañero.

2. Obligaciones del enunciado que guían el reparto

Exigencia del profesor	Dónde la resolvemos
Frontend React y backend API independientes	P3/P4 frontend; P1/P2 Quiz API
Autenticación y perfiles	P3 integra login, P1 protege API; decisión Supabase pendiente
CRUD de cuestionarios/preguntas con imágenes	P1 backend + P3 interfaz; persistencia verificada
Salas separadas, código, anfitrión y participantes	P2 motor + P3/P4 pantallas
Tiempo real y autenticación del canal	P2 Gateway + P4 cliente + P1 JWT guard
Una respuesta, tiempo límite, revelación y puntaje	P2 motor; P1 restricciones DB; P4 experiencia participante
Reconexión sin duplicar puntaje	P2 estado/identidad; P4 reconexión cliente
REST independiente + cliente adicional	P1/P2 OpenAPI; P4 cliente lector
OpenSpec, Scalar, unitarias/e2e	Documentación por dueño; pruebas cruzadas
Docker, Compose, BD, imágenes, Docker Hub y URL pública	P4 coordina, P1 DB/migraciones, P2 runtime, P3 frontend
OpenTelemetry + Grafana para APIs	P1/P2 instrumentación, P4 verifica dashboards
Partida desde varios dispositivos y medición de participantes	Prueba final de todo el equipo

Nota de alcance: la página 4 del enunciado enumera webhooks de cierre de partida, colas y tareas en segundo plano como extensiones opcionales. No deben desplazar la funcionalidad obligatoria. La instrumentación OpenTelemetry/Grafana sí aparece expresamente como requisito de servicios en la página 2.

3. Decisiones base y contratos comunes

3.1 Arquitectura propuesta

Una sola API propia del juego (NestJS, monolito modular organizado en capas) más un frontend React. Supabase Auth como servicio externo de identidad. PostgreSQL almacena información del juego; las imágenes van a almacenamiento persistente. NestJS expone REST para operaciones y Socket.IO para eventos en vivo. La API nunca confía en acciones privilegiadas ni en tiempos declarados por React.

3.2 Modelo de datos mínimo a acordar

Entidad	Campos/relaciones esenciales	Regla
Profile	userId (UUID de Supabase), displayName	No guardar contraseñas
Quiz	id, ownerId, title, updatedAt	Solo el propietario modifica
Question	id, quizId, statement, correctBoolean, imageKey?, order	Imagen validada; orden estable
GameSession	id, code, quizId, hostId, status, currentIndex, timestamps	Código no predecible y único
SessionQuestion	id, sessionId, originalQuestionId, snapshot de contenido/respuesta	Congela el quiz durante la partida
SessionParticipant	sessionId, userId, joinedAt, displayName	UNIQUE(sessionId,userId)
Answer	sessionQuestionId, participantId, choice, answeredAt, score	UNIQUE(sessionQuestionId,participantId)
GameResult	sessionId, participantId, totalScore, rank	Persistencia al finalizar

Las columnas exactas, índices y nombres de tablas se formalizan con el equipo. La idea fundamental es impedir duplicados en la base de datos, no solo con una variable del frontend. Los perfiles pueden usar el UUID de Supabase sin copiar contraseñas.

3.3 Estados y seguridad comunes

Concepto	Estados/regla acordada
Sala	WAITING → IN_PROGRESS → FINISHED
Pregunta	OPEN → CLOSED; no revelar correctBoolean mientras esté OPEN
Puntaje	Acierto = 1; error o falta de respuesta = 0
Tiempo	openedAt / closesAt del servidor; respuesta tardía se rechaza
Permisos	Host controla su sala; participante solo responde en su propia sala
Reconexión	Verificar JWT; upsert lógico de participante; enviar snapshot del estado
Aislamiento	Room de Socket.IO por sessionId y filtrado de consultas por sesión

3.4 Contratos REST y eventos (borrador)

REST / evento	Responsable	Qué entrega
GET /profiles/me	P1	Perfil del usuario autenticado
POST/GET/PATCH/DELETE /quizzes	P1	CRUD con ownership
POST /quizzes/:id/questions	P1	Creación de pregunta con referencia a imagen
POST /sessions	P2	Código y estado WAITING
POST /sessions/:code/join	P2	Participación única en sala
GET /sessions/:code/state	P2	Estado para reconexión/cliente extra
GET /sessions/:id/results	P2	Clasificación autorizada
room:join / room:state	P2+P4	Suscripción y snapshot
game:start / question:opened	P2+P3+P4	Inicio y pregunta para jugadores
answer:submit / answer:ack	P2+P4	Respuesta aceptada/rechazada
question:closed / game:finished	P2+P3+P4	Corrección y clasificación

Antes de escribir pantallas
Crear dos ejemplos JSON por contrato: éxito y error. Decidir estructura única de errores: código, mensaje, detalles y estado HTTP. Congelar los nombres de eventos para que React y NestJS avancen en paralelo.

4. Persona 1 — Base de datos, API REST e imágenes

Misión: Proveer datos confiables y endpoints que el resto pueda consumir: perfiles, cuestionarios, preguntas y archivos.

Analogía: Es el encargado del archivo y las reglas de acceso: debe asegurar que cada profesor edite solo su material y que los documentos nunca se pierdan.

Es responsable de: Modelo relacional, migraciones, TypeORM, REST, validación de JWT en HTTP, ownership, almacenamiento de imágenes, pruebas unitarias del módulo.

No es responsable de: Motor de temporizadores, comunicación Socket.IO y vista visual del anfitrión (excepto contratos compartidos).

Paso 1. Preparar DB y migraciones

Qué hacer: Instalar el proyecto NestJS acordado y TypeORM, conexión PostgreSQL por variables, primera migración. Crear perfiles y tablas de contenido con claves foráneas.

Cómo comprobarlo: En una BD limpia: aplicar migración, reiniciar API y leer un perfil/quiz; no depende de datos precargados a mano.

Paso 2. Integrar identidad de Supabase

Qué hacer: Acordar con P3 cómo se obtiene un JWT; crear guard que verifica firma, issuer/exp y claim de usuario. Validar audiencia según autorización del profesor; no usar JWT simplemente decodificado.

Cómo comprobarlo: Sin token → 401; token inválido → 401; token correcto → perfil propio. Nunca se filtran datos de otros usuarios.

Paso 3. Implementar perfiles

Qué hacer: GET /profiles/me; guardar/actualizar displayName ligado al userId del token. Definir DTO y errores.

Cómo comprobarlo: La cuenta A ve su nombre; la cuenta B no puede editar el perfil A.

Paso 4. Crear CRUD de cuestionarios

Qué hacer: DTO, validación, persistencia, propietario y rutas GET/POST/PATCH/DELETE. Decidir qué sucede si un quiz ya tiene una partida.

Cómo comprobarlo: Crear, listar, editar, borrar con pruebas; otro usuario no edita el quiz ajeno.

Paso 5. Agregar preguntas

Qué hacer: Enunciado, correctBoolean, orden, imagen opcional. Validar campos; prohibir modificaciones a copias de preguntas ya usadas en una sesión.

Cómo comprobarlo: Un quiz mantiene orden; V/F se valida en backend; durante partida se conserva snapshot.

Paso 6. Subida de imágenes

Qué hacer: Definir formato permitido y límite de tamaño; almacenar en Supabase Storage o adaptador persistente. Guardar solo URL/clave de objeto en BD. Proteger subida y borrado.

Cómo comprobarlo: Se sube imagen, aparece en otro dispositivo, persiste al reiniciar; archivo no permitido se rechaza.

Paso 7. Contratos, Scalar y pruebas

Qué hacer: Publicar OpenAPI/Scalar y ejemplos; pruebas para permisos, campos inválidos y datos de base. Acordar modelo Session/Answer con P2 y revisar sus restricciones únicas.

Cómo comprobarlo: P3 trabaja contra estos endpoints; P2 puede crear sesiones usando entidades ya migradas.

Paso 8. Entrega e integración

Qué hacer: Preparar seed de prueba e instrucciones DB, OpenSpec archivado en PR y revisión cruzada con P3/P2.

Cómo comprobarlo: Demo: login → crear quiz con pregunta/imagen → otra cuenta no puede editar → DB conserva los datos.

Entregables y dependencia crítica de P1

- Migraciones versionadas, modelo de datos y diagrama ER/clases; no modificaciones manuales ocultas en Supabase.
- API REST segura, OpenAPI/Scalar, archivos de prueba y seed reproducible.
- Entrega temprana a P3: endpoints de perfiles/cuestionarios/preguntas + JSON de ejemplo.
- Entrega temprana a P2: entidades GameSession, SessionParticipant, SessionQuestion y Answer, o migraciones acordadas en conjunto.

5. Persona 2 — Motor del juego, salas y WebSockets

Misión: Garantizar una partida justa, simultánea, aislada y recuperable tras desconexiones.

Analogía: Es el árbitro: el cronómetro, el permiso para responder y el marcador están bajo su control, no en el celular del jugador.

Es responsable de: Reglas de estado, API de sesiones y resultados, WebSocket Gateway, autorización de eventos, temporizadores, respuesta única, reconexión del servidor y pruebas concurrentes.

No es responsable de: CRUD de cuestionarios o pantallas React; P1 proporciona migraciones y P4 implementa la reconexión visual.

Paso 1. Escribir máquina de estados

Qué hacer: Definir transiciones permitidas para sala y pregunta; reglas de host y cierre, qué ocurre al finalizar y qué se persiste.

Cómo comprobarlo: Una solicitud fuera de orden devuelve error claro; nunca se inicia dos veces una partida.

Paso 2. Crear y unir sala por REST

Qué hacer: POST /sessions, código único, GET estado, join de participante autenticado. Usar unicidad (sessionId, userId).

Cómo comprobarlo: Dos salas creadas no comparten jugadores ni códigos; segundo join del mismo usuario no duplica registros.

Paso 3. Integrar Gateway Socket.IO

Qué hacer: Definir autenticación JWT en handshake/eventos, rooms por sessionId, host vs participante, eventos acordados y respuestas ack.

Cómo comprobarlo: Una cuenta no puede publicar game:start de una sala ajena; ningún broadcast llega a otra sala.

Paso 4. Enviar preguntas seguras

Qué hacer: Publicar enunciado, imagen y tiempo restante, sin correctBoolean mientras esté abierta. Tomar snapshot antes de iniciar partida.

Cómo comprobarlo: Dos dispositivos muestran la misma pregunta; cambiar el quiz original no afecta la partida.

Paso 5. Validar respuestas y tiempos en servidor

Qué hacer: Recibir choice, sessionQuestionId y userId desde el token; verificar sala, pertenencia, estado OPEN, closesAt y unicidad BD. Insertar una sola Answer.

Cómo comprobarlo: Doble clic/concurrencia no produce dos registros; respuesta tardía se rechaza; cliente no puede responder por otro usuario.

Paso 6. Cerrar y puntuar

Qué hacer: Tras cierre, calcular 1/0, publicar respuesta correcta y avance autorizado por anfitrión. Acumular resultado sin duplicar sumas.

Cómo comprobarlo: Reenvío del mismo evento no aumenta puntaje; todos ven la corrección después del cierre.

Paso 7. Reconexión y reinicios

Qué hacer: Implementar GET /sessions/:code/state y socket room:join que devuelven snapshot actual; recuperar participación por identidad persistida. Documentar comportamiento ante reinicio de servidor durante una partida.

Cómo comprobarlo: Desconectar un jugador, reconectar y mostrar mismo usuario y puntaje, sin registro adicional.

Paso 8. Pruebas multiusuario y telemetría

Qué hacer: Pruebas de concurrencia, 2 salas paralelas, tiempos, permisos, recuperación. Instrumentación de contadores/latencia útiles para Grafana.

Cómo comprobarlo: Demostración reproducible con varios dispositivos y registro real de máximo concurrente probado.

Entregables y dependencia crítica de P2

- Motor sin reglas de negocio en el Gateway: servicio de dominio que valida, puntúa y transiciona estados.
- Contratos REST y WebSocket con estructura exacta de payload, ack y errores; listado de códigos de error.

- Entrega temprana a P3: game:start, question:opened y permisos del host; a P4: room:join, answer:submit, answer:ack, room:state.
- Pruebas de una respuesta válida, duplicada, tardía, sin permiso y de dos salas simultáneas.

6. Persona 3 — Frontend React: identidad, cuestionarios y anfitrión

Misión: Permitir que el profesor/organizador prepare contenido, abra una sala y dirija la actividad sin complejidad.

Analogía: Es quien diseña la mesa de control: muestra al anfitrión las herramientas, pero la autorización y las reglas viven en el backend.

Es responsable de: Autenticación visual Supabase, guardado de sesión en navegador, pantallas de perfiles/quiz/preguntas/imágenes, lobby y control del anfitrión, pruebas de interacción.

No es responsable de: Validar puntajes, decidir quién ganó o cerrar preguntas mediante reloj del navegador; eso lo determina P2.

Paso 1. Montar React y componentes compartidos

Qué hacer: Rutas, layout responsive, cliente HTTP, control de carga/errores y componentes reutilizables. Acordar estilos con P4.

Cómo comprobarlo: Navegación desde móvil y PC; pantallas iniciales accesibles y consistentes.

Paso 2. Login y sesión Supabase

Qué hacer: Registro, login, logout, nombre visible y recuperación de sesión con supabase-js. Inyectar JWT en peticiones HTTP; coordinar criterio de audiencia con P1.

Cómo comprobarlo: Recargar página no cierra sesión inesperadamente; rutas privadas bloqueadas sin autenticación.

Paso 3. Panel de cuestionarios

Qué hacer: Listar propios, crear, editar, borrar y ver detalle. Usar DTO y respuestas reales de P1 o mocks fieles al contrato.

Cómo comprobarlo: CRUD completo, mensajes de validación y permiso negado visibles.

Paso 4. Editor de preguntas e imagen

Qué hacer: Interfaz para texto/VF, orden, vista previa y subida de archivo. Mostrar restricciones de formato y tamaño antes de subir, sin confiar solo en React.

Cómo comprobarlo: Pregunta persiste con imagen; se muestra en otro equipo; error de archivo inválido es comprensible.

Paso 5. Crear sala como anfitrión

Qué hacer: Seleccionar quiz, POST /sessions, mostrar código/URL, lista de participantes y botón de iniciar.

Cómo comprobarlo: Código real, participantes se actualizan vía eventos y sólo el host puede iniciar.

Paso 6. Pantalla en vivo del host

Qué hacer: Escuchar eventos de P2: estado, participantes, preguntas, cierre, resultados. Permitir avanzar cuando backend lo autorice.

Cómo comprobarlo: El host no revela respuestas antes de tiempo ni altera estados mediante cambios de frontend.

Paso 7. Manejo de errores y reconexión

Qué hacer: Mostrar «sin conexión», restaurando estado y mensajes si la sala cierra/ya inició/código es inválido. Coordinación UX con P4.

Cómo comprobarlo: Al reconectar no aparece una partida duplicada; el estado coincide con el backend.

Paso 8. Pruebas E2E y entrega

Qué hacer: Automatizar al menos registro/login y creación de quiz/sala cuando el entorno de pruebas esté listo. Capturas desktop/móvil, OpenSpec de pantallas/contratos consumidos.

Cómo comprobarlo: Recorrido host: login → quiz → imagen → sala → inicia → termina → ve clasificación.

Entregables y dependencia crítica de P3

- Pantallas React responsivas: registro, login, editor, preguntas con imagen, lobby y sala del anfitrión.
- Cliente HTTP estándar que P4 reutiliza; manejo de JWT común, sin duplicar cliente Supabase.
- Entrega temprana a P4: estructura React, rutas, componentes, hook de usuario y diseño visual.
- Manual breve del flujo del anfitrión y pruebas E2E de interfaz.

7. Persona 4 — Frontend participante, reconexión e integración

Misión: Lograr que alumnos reales puedan unirse y responder desde su teléfono y que el sistema completo sea fácil de ejecutar por otra persona.

Analogía: Es el encargado de que el público pueda entrar al estadio y que las puertas, marcadores, cámaras y conexión funcionen el día del evento.

Es responsable de: Pantallas participantes, Socket.IO cliente, reconexión y errores, cliente independiente de API, coordinación de Docker/Compose, publicación y prueba integral.

No es responsable de: Implementar reglas de puntuación o modificar eventos sin acuerdo con P2; infraestructura es compartida, no trabajo exclusivo de P4.

Paso 1. Reutilizar cliente React compartido

Qué hacer: Acordar rutas, sesión Supabase, diseño y convenciones con P3. Configurar el cliente Socket.IO con token autenticado.

Cómo comprobarlo: Mismo usuario y estilos entre vista de profesor y vista de participante.

Paso 2. Unirse por código

Qué hacer: Pantalla de código, validaciones, GET/POST REST, pantalla de espera y lista/estado de participantes.

Cómo comprobarlo: Con código inválido aparece error; unirse dos veces no duplica al jugador.

Paso 3. Responder pregunta en tiempo real

Qué hacer: Mostrar pregunta e imagen desde evento question:opened y botones V/F; enviar answer:submit con ack. Desactivar botones al aceptar la respuesta.

Cómo comprobarlo: Doble clic no crea respuesta extra; la confirmación visible coincide con backend.

Paso 4. Mostrar cierre y clasificación

Qué hacer: Pantalla de tiempo restante informativo, corrección después de cierre, puntaje y resultado final.

Cómo comprobarlo: No muestra respuesta correcta antes del cierre; clasificación coincide con API.

Paso 5. Reconectarse sin duplicarse

Qué hacer: Mostrar estado de conexión; revalidar sesión, volver a room:join y usar snapshot room:state. No depender del estado anterior de React para marcar preguntas como contestadas.

Cómo comprobarlo: Cerrar red unos segundos, reconectar y recuperar pregunta/estado y puntaje.

Paso 6. Cliente adicional independiente

Qué hacer: Crear página pequeña o script/cliente distinto de React principal que consuma REST con token y consulte estado o resultados sin acceder a la DB.

Cómo comprobarlo: Abrir en un navegador/equipo separado y mostrar respuesta de API documentada.

Paso 7. Docker y despliegue con apoyo

Qué hacer: Coordinar Dockerfile frontend/API, Compose con PostgreSQL, volúmenes, .env.example, migraciones/seed, Docker Hub, script de inicio y URL HTTPS. P1 prepara DB, P2 servicio runtime, P3 frontend.

Cómo comprobarlo: Clonar en instalación limpia, completar variables, ejecutar script, crear usuario/quiz demo, jugar desde móviles.

Paso 8. Prueba de clase y manual

Qué hacer: Coordinar pruebas multi-dispositivo y límite medido, monitoreo Grafana con P1/P2, guía de demo, capturas y problemas detectados.

Cómo comprobarlo: Existe registro del máximo de jugadores realmente probado y de dos salas que no mezclan eventos.

Entregables y dependencia crítica de P4

- Pantallas de jugador completas y reconexión visible probada contra P2.
- Cliente adicional que funciona por REST con configuración independiente y autenticación.
- Compose, script y README reproducibles, imágenes Docker Hub y URL desplegada; con commits concretos de soporte de P1-P3.
- Prueba integral documentada, con cantidad real de participantes probados y limitaciones observadas.

8. Orden de desarrollo y entregas pequeñas

Hito	Primero se construye	Evidencia que habilita el siguiente
0. Acuerdos	Confirmar interpretación Supabase; stack, roles, datos, diagramas, contratos, Git/OpenSpec	Dos cuentas y JWT válidos; especificación y PR de proyecto
1. Vertical mínimo	Auth React → API protegida → DB → crear un quiz simple	El creador puede crear un quiz y verlo al recargar
2. Sala y comunicación	Crear sala REST → join → Gateway auth → emisión de una pregunta	Dos móviles reciben la misma pregunta en una sala
3. Respuesta confiable	Una respuesta, tiempo, cierre y puntaje; frontend participante	Duplicados/tardías no suman; se publica resultado
4. Resiliencia	Varias salas, reconexión, snapshot y persistencia de resultados	Jugador vuelve sin duplicado; salas aisladas
5. Imagen y consumo	Carga persistente, imagen visible, cliente REST adicional	Otro dispositivo ve imagen; cliente externo consulta resultados
6. Entrega real	Compose limpio, imágenes Hub, cloud, Grafana, pruebas, README	Grupo completo juega en varios móviles y defiende evidencias

Primera integración obligatoria (no esperar semanas)

1. P1 crea la migración mínima Profile/Quiz y el endpoint protegido POST /quizzes.
2. P3 inicia sesión con Supabase desde React y manda el JWT real al endpoint.
3. P2 incorpora POST /sessions y un evento Socket.IO «room:state» protegido.
4. P4 abre dos clientes React y verifica unión/reconexión a la misma sala.
5. El grupo hace una PR de integración con pruebas y documentación. Solo después amplía el juego.

9. Dependencias y pactos entre integrantes

Entrega de	Para	Debe contener	Fecha relativa
P1 → P3	Frontend autor	JWT guard y contrato CRUD + ejemplos de errores	Hito 1
P1 → P2	Motor de juego	Modelo Session/Question/Answer y migraciones	Antes de hito 2
P2 → P3	Host	Eventos start/open/close/finish y autorizaciones	Hito 2
P2 → P4	Participante	Eventos join/answer/ack/state, manejo de reconexión	Hitos 2-3
P3 → P4	Frontend compartido	Login, hook usuario, HTTP y componentes visuales	Hito 1
P1/P2/P3 → P4	Docker e integración	Dockerfile, variables, healthcheck, comandos de inicio	Antes de hito 6

Cómo resolver bloqueos

- Si falta el endpoint, la persona frontend usa un mock que cumpla exactamente el JSON acordado; no inventa otro formato.
- Si falta la pantalla, la persona backend prueba con curl/cliente WebSocket y deja casos reproducibles.
- Si cambia una entidad o evento, abrir primero PR de contrato y acordarlo; después implementar versiones correspondientes.
- Si un trabajo supera el esfuerzo esperado, el equipo reasigna una subtarea concreta (por ejemplo pruebas/migración/UI) sin quitar la propiedad del módulo.

10. Protocolo Git, OpenSpec y revisión

1. Crear rama pequeña por tarea: feat/quiz-create, feat/room-join, feat/host-panel, feat/player-reconnect.
2. Antes de implementar, agregar/actualizar el cambio OpenSpec: propósito, criterios de aceptación, contratos y reglas; acordar con consumidor.
3. Escribir código + pruebas + README localizado. No subir .env, tokens, llaves ni contraseñas.
4. Abrir Pull Request con objetivo, qué hace, pruebas ejecutadas, capturas, variables agregadas y limitaciones.
5. Adjuntar versión finalizada/archivada del spec (opsx archive / opsx-archive según herramienta instalada), tal como exige el enunciado; comprobar nombre/comando instalado.
6. Un compañero revisa y ejecuta una prueba. Si cambió REST o eventos, requieren aprobación de consumidor.
7. Fusionar a rama de integración/branch principal solo con build y pruebas sin fallos. Actualizar documentación y ejemplos.

Plantilla corta para cada tarea

Tarjeta de tarea
ID y responsable | Objetivo | Archivos/módulos | Dependencias | Contrato (ruta/evento) | Pasos 1/2/3 | Casos correcto/incorrecto | Pruebas | Entregable | Estado: pendiente/en curso/revisión/hecho.

11. Pruebas mínimas, casos adversos y criterios

Escenario	Quién lidera	Qué se considera éxito
Sin JWT / JWT inválido / otro propietario	P1	401/403 según caso; no se exponen datos
Crear/editar/borrar quiz con preguntas e imagen	P1 + P3	CRUD válido y persistente; formato/tamaño inválido rechazado
Dos salas a la vez	P2 + P4	No se cruzan eventos ni participantes
Respuesta duplicada en paralelo	P2 + P1	Una respuesta persistida y un solo punto
Respuesta enviada después de closesAt	P2	Se rechaza sin afectar resultados
Participante intenta game:start	P2 + P4	Evento denegado incluso por socket manual
Reconexión tras cerrar Wi-Fi	P2 + P4	Mismo participantId/puntaje y estado restaurado
Quiz original editado durante sesión	P1 + P2	Snapshot de sesión no cambia
Reiniciar contenedores	P1 + P4	Perfiles/quizzes/imágenes/resultados finalizados sobreviven
Cliente adicional REST	P4 + P1	Consulta sesión/resultados sin acceso a BD
Prueba de clase con varios dispositivos	Todos	Registrar cantidad, condiciones y fallos reales
Telemetry/API	P1 + P2	Métricas HTTP y juego aparecen, dashboards verificables

12. Docker, nube y persistencia

Contrato de ejecución reproducible

- Docker Compose debe arrancar las aplicaciones propias y PostgreSQL con volumen; la API debe aplicar migraciones y sembrar datos de demostración de forma controlada.
- La dependencia Supabase Auth/Storage es externa si se mantiene Cloud: el README debe explicarlo, pedir credenciales de prueba y aclarar que sin conectividad no existe login. Si se exige autonomía completamente local, habrá que diseñar variante local y presupuestar el esfuerzo.
- Cada aplicación tiene Dockerfile; Compose configura healthchecks, URLs, puertos, secretos por .env y arranque reproducible.
- Publicar imágenes del equipo en Docker Hub y comprobar «docker compose pull + up» desde una máquina distinta y limpia.
- Para imágenes, usar Storage remoto persistente o un almacenamiento con volumen definido; nunca solo carpeta efímera del contenedor.
- Desplegar URL HTTPS externa, probar desde celular fuera de la máquina del desarrollador; no asumir que localhost es accesible desde el aula.

Responsables de infraestructura compartida

Componente	Implementa	Verifica
PostgreSQL, migraciones y seed	P1	P2 y P4
Dockerfile Quiz API y salud/variables	P2 con P1	P4
Dockerfile frontend/configuración API	P3	P4

Componente	Implementa	Verifica
Compose, script, Docker Hub y README	P4 con soporte de P1-P3	Todo el equipo
OpenTelemetry/servicios y Grafana	P1+P2	P4 y P3

13. Plan para la exposición y criterios de defensa

Recorrido único de demostración

1. Abrir la URL HTTPS del frontend en el proyector. Iniciar sesión y mostrar un perfil.
2. El anfitrión crea quiz de varias preguntas V/F; una incluye imagen persistida.
3. Crear sala y mostrar código. Al menos dos compañeros se unen desde móviles; observar lobby en tiempo real.
4. Iniciar partida: proyectar pregunta y responder desde dispositivos; el servidor valida tiempo y unicidad.
5. Cerrar pregunta, revelar respuesta y puntajes; mostrar clasificación final y resultados persistidos.
6. Desconectar/reconectar un participante: vuelve al mismo estado sin duplicar registros.
7. Abrir una segunda sala y demostrar que participantes/resultados no se cruzan.
8. Abrir cliente adicional REST y consultar resultados con token sin acceder a DB.
9. Mostrar OpenAPI/Scalar de las APIs relevantes, una prueba de fallo de permisos, Compose/Docker Hub y panel Grafana.
10. Enseñar evidencia de máximo de participantes probado, límites observados y reparto de aportes en PRs.

Qué debe explicar cada integrante sin ayuda

Persona	Preguntas que debe poder contestar
P1	¿Cómo se persisten los quizzes? ¿Cómo se valida JWT y propiedad? ¿Por qué migraciones y restricciones únicas? ¿Dónde queda la imagen?
P2	¿Cómo viaja la pregunta? ¿Por qué el servidor controla tiempo/puntaje? ¿Cómo evitamos duplicados, mezcla de salas y reconexiones erróneas?
P3	¿Cómo se inicia sesión? ¿Qué operaciones puede ejecutar un anfitrión? ¿Cómo se muestran los errores del backend?
P4	¿Cómo responde y se reconecta un participante? ¿Cómo corre un proyecto externo usando la API? ¿Cómo se replica Docker y se prueba en móviles?

14. Riesgos y plan de prevención

Riesgo	Cómo reducirlo
Supabase Auth no satisface la API independiente	Confirmación expresa del profesor antes de fijar contratos; si exige API separada, planificar un servicio propio delgado con NestJS.
Cambios de DTO y eventos al final	Contrato OpenSpec y ejemplos JSON desde el hito 0; integración vertical muy temprano.
P2 termina sobrecargado	P1 hace restricciones y migraciones, P4 cliente/reconexión visual, P3 UI host; pruebas compartidas.
Falla de red el día de exposición	Ensayar conectividad HTTPS desde móviles y preparar

Riesgo	Cómo reducirlo
	demostración local con servicios/documentación; no mostrar como real una prueba no ejecutada.
Mensajes llegan a sala incorrecta	Aislar room por id, autorizar pertenencia en cada evento y probar dos salas.
Duplicados dan puntos adicionales	Restricción única DB + lógica servidor idempotente + transacción + prueba concurrente.
Se pierde progreso al reiniciar	Persistir resultados finales y datos importantes, snapshots y tiempo del servidor; documentar alcance de recuperación de partida activa.
Secretos/keys se filtraron	Solo variables de entorno, .gitignore, .env.example sin secretos, revisión del repositorio.
Costo de servicios gratuitos	Probar límites reales y alternativas, y verificar que cuentas/servicios estén activos antes de demo.

15. Checklist de cierre por persona

P1 — API REST y datos

☐ Migraciones y seed; diagrama ER/clases

☐ JWT y ownership verificados con 401/403

☐ CRUD e imágenes probados desde otro dispositivo

☐ Scalar/OpenSpec y PRs con pruebas

P2 — Juego y tiempo real

☐ Salas únicas aisladas + autenticación Socket.IO

☐ Respuesta única, límite de tiempo, puntuación y cierre

☐ Reconexión/estado y resultados persistidos

☐ Pruebas concurrentes y métricas del juego

P3 — Frontend host

☐ Login Supabase y navegación segura

☐ Crear quiz y subir imagen

☐ Crear sala, esperar participantes y dirigir partida

☐ Prueba E2E + respuesta clara a fallos

P4 — Participante e integración

☐ Join, respuesta, resultado y reconexión demostrados

☐ Cliente REST extra autenticado

☐ Compose, Docker Hub, URL y README reproducibles

☐ Prueba varios móviles + registro de máxima concurrencia

Chequeo colectivo final

☐ Cada integrante tiene commits, PRs, revisiones y aportes reconocibles.

☐ La partida completa funciona desde una URL pública HTTPS.

☐ El README contiene instrucciones desde cero, secretos de ejemplo, arranque y detención.

☐ El grupo puede repetir la demo y responder preguntas sobre cualquier módulo.

☐ Se entregan especificaciones, diagramas, API/eventos, pruebas y evidencia de límite real medido.

16. Mensaje corto para compartir al grupo

Compañeros, dividiremos ProfeLink en cuatro áreas: (1) backend REST y base de datos, (2) motor del juego y WebSockets, (3) frontend del creador/anfitrión y autenticación Supabase, y (4) frontend del participante e integración final. Todos debemos programar una parte funcional, escribir pruebas, documentar los contratos OpenSpec y ayudar con Docker, Grafana y la exposición. Primero acordaremos las entidades, rutas, eventos y estados; después trabajaremos en PRs pequeños y probaremos una integración mínima lo antes posible. Nadie cambia contratos sin consultarlo. Una tarea solo estará terminada si funciona, tiene pruebas, documentación y fue integrada. El objetivo es que cualquiera de nosotros pueda explicar el flujo completo desde login hasta resultados y reconexión.

Fuente y naturaleza de las decisiones

BASE DEL ENUNCIADO. “Proyecto Final Parte 2 — Evaluaciones en vivo para ProfeLink”, Paradigmas de Programación EIF400, II Ciclo 2026, profesor M.S. Alex Villegas Carranza, 4 páginas, entregado por el usuario. Requisitos: páginas 1-3; criterios, defensa y opcionales: página 4.

ACUERDOS DEL EQUIPO. Arquitectura simplificada React + NestJS + Supabase Auth, división de cuatro áreas y estilo de planificación incremental provienen de la conversación; son propuestas organizativas, no obligaciones literales del profesor.

ALCANCE PENDIENTE. Confirmar ante el profesor si Supabase Auth como servicio externo cubre la obligación específica de una API de autenticación desplegada aparte y audiencia; y acordar solución de imágenes y ejecución local reproducible.